UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Faculty of Computing

PROJECT PROGRESS 2

Subject: Programming For Bioinformatics (SECB3202)

Session: 2025/2026 Semester 1

Title: Pancreatic Cancer Prediction

Group Name	Group 10
Group Members	1. RAVINESH A/L MARAN (A23CS0175) 2. WELSON WOONG LU BIN (A23CS0196) 3. BERNICE LOU MIN YUN (A23CS0056)
Section	01
Lecturer's Name	DR. SEAH CHOON SEN

Table of Content

1.0Importing Dataset

1.1 Understanding the Dataset

1.2 Importing and Exporting Data in Python

1.3 Getting started and analyzing data in Python

1.4 Python Packages for Data Science

2.0 Data Wrangling (Pandas/ Numpy)

2.1 Identifying and handling missing value

2.2 Data Formatting

2.3 Data Normalization (centering/ scaling)

2.4 Binning

2.5 Indicator Variables

1.0 Importing Dataset

1.1 Understanding the Dataset

The dataset used in this project is a Pancreatic Cancer Prediction dataset, which contains biomarker measurements and patient-related attributes relevant to disease classification. The dataset is intended to support the prediction of pancreatic cancer using machine learning techniques.

Before proceeding with data wrangling and model development, it is essential to understand the dataset structure, data types, and completeness. This step allows early identification of potential issues such as missing values, inconsistent data types, and abnormal distributions that may affect subsequent analysis.

1.2 Importing and Exporting Data in Python

The dataset was stored in CSV format and imported into the Python environment using the Pandas library within the Visual Studio Code platform. Pandas provides efficient functions for loading, handling, and manipulating structured data, making it suitable for bioinformatics-related datasets.

Importing the dataset into a Pandas DataFrame enables flexible data inspection, preprocessing, and transformation prior to analysis.

```
df = pd.read_csv(r"C:\bioinfo2\p4b\project\Pancrease_Cancer_Prediction\pancreatic_cancer_prediction_sample.csv")
df.head()
```

```
df = pd.read_csv(r"C:\bioinfo2\p4b\project\Pancrease_Cancer_Prediction\pancreatic_cancer_prediction_sample.csv")
df.head()
```

✓ 0.1s Python

	Country	Age	Gender	Smoking_History	Obesity	Diabetes	Chronic_Pancreatitis	Family_History	Hereditary_Condition	Jaundice	...	Stage_at_Diagnosis	Survival_Time_Months	Treatmen
0	Canada	64	Female	0	0	0	0	0	0	0	...	Stage III	13	
1	South Africa	77	Male	1	1	0	0	0	0	0	...	Stage III	13	Chemot
2	India	71	Female	0	0	0	0	0	0	0	...	Stage IV	3	Chemot
3	Germany	56	Male	0	0	0	0	1	0	1	...	Stage IV	6	Ra
4	United States	82	Female	0	0	0	0	1	0	0	...	Stage IV	9	Chemot

5 rows × 24 columns

1.3 Getting started and analyzing data in Python

After importing the dataset, an initial analysis was conducted to gain an overview of both numerical and categorical features using Python. Descriptive statistical methods were applied to summarise the data and to understand the distribution, variability, and characteristics of the dataset before proceeding with further preprocessing and modeling.

```
df.describe().T
```
✓ 0.0s

	count	mean	std	min	25%	50%	75%	max
Age	50000.0	64.54094	9.973847	30.0	58.0	65.0	71.0	90.0
Smoking_History	50000.0	0.29954	0.458061	0.0	0.0	0.0	1.0	1.0
Obesity	50000.0	0.24826	0.432008	0.0	0.0	0.0	0.0	1.0
Diabetes	50000.0	0.19998	0.399989	0.0	0.0	0.0	0.0	1.0
Chronic_Pancreatitis	50000.0	0.09930	0.299067	0.0	0.0	0.0	0.0	1.0
Family_History	50000.0	0.15168	0.358714	0.0	0.0	0.0	0.0	1.0
Hereditary_Condition	50000.0	0.04944	0.216787	0.0	0.0	0.0	0.0	1.0
Jaundice	50000.0	0.19922	0.399418	0.0	0.0	0.0	0.0	1.0
Abdominal_Discomfort	50000.0	0.29650	0.456719	0.0	0.0	0.0	1.0	1.0
Back_Pain	50000.0	0.25286	0.434656	0.0	0.0	0.0	1.0	1.0
Weight_Loss	50000.0	0.34998	0.476968	0.0	0.0	0.0	1.0	1.0
Development_of_Type2_Diabetes	50000.0	0.19622	0.397141	0.0	0.0	0.0	0.0	1.0
Survival_Time_Months	50000.0	13.89804	11.272151	1.0	6.0	10.0	19.0	59.0
Survival_Status	50000.0	0.12844	0.334582	0.0	0.0	0.0	0.0	1.0
Alcohol_Consumption	50000.0	0.30346	0.459757	0.0	0.0	0.0	1.0	1.0

The df.describe( ) function was used to generate descriptive statistics for numerical variables in the dataset. This includes measures such as count, mean, standard deviation, minimum and maximum values, as well as quartiles (25%, 50%, and 75%).

From the results, the dataset contains 50,000 records, indicating a large and reliable sample size. The average age of patients is approximately 64.5 years, with most patients falling between 58 and 71 years, suggesting that pancreatic cancer predominantly affects older individuals. Survival time statistics show a median of 10

months, reflecting the aggressive nature of the disease and generally low survival outcomes.

Several clinical and lifestyle factors, such as smoking history, obesity, diabetes, and alcohol consumption, are represented as binary indicators. The mean values of these variables represent their prevalence in the dataset. For example, smoking history and alcohol consumption are present in roughly 30% of patients. This numerical analysis helps identify the distribution of key risk factors within the population.

```
df.describe(include='object').T
```

✓ 0.0s

	count	unique	top	freq
Country	50000	9	United States	17608
Gender	50000	2	Male	25962
Stage_at_Diagnosis	50000	4	Stage IV	19922
Treatment_Type	50000	3	Chemotherapy	24910
Physical_Activity_Level	50000	3	Medium	20038
Diet_Processed_Food	50000	3	Medium	20122
Access_to_Healthcare	50000	3	Medium	25268
Urban_vs_Rural	50000	2	Urban	35003
Economic_Status	50000	3	Middle	24881

The df.describe(include='object') function was applied to analyse categorical variables by examining their counts, number of unique categories, most frequent values, and frequency of occurrence.

The results show that the majority of patients are from the United States, and male patients slightly outnumber female patients. A large proportion of patients are diagnosed at Stage IV, indicating late-stage detection of pancreatic cancer.

Chemotherapy is the most commonly recorded treatment type, which aligns with clinical practice for advanced-stage cases.

Lifestyle and socio-economic variables, including physical activity level, diet, access to healthcare, residential setting, and economic status, are predominantly classified as "Medium," suggesting moderate lifestyle and healthcare conditions among most patients. This categorical analysis provides valuable insight into the demographic and clinical composition of the dataset.

1.4 Python Packages for Data Science

Several Python libraries were utilized during this phase of the project:

- Pandas – data loading, manipulation, and preprocessing
- NumPy – numerical computations and array operations
- Matplotlib – basic data visualization
- Seaborn – statistical data visualization
- Scikit-learn – data normalization and preparation for machine learning

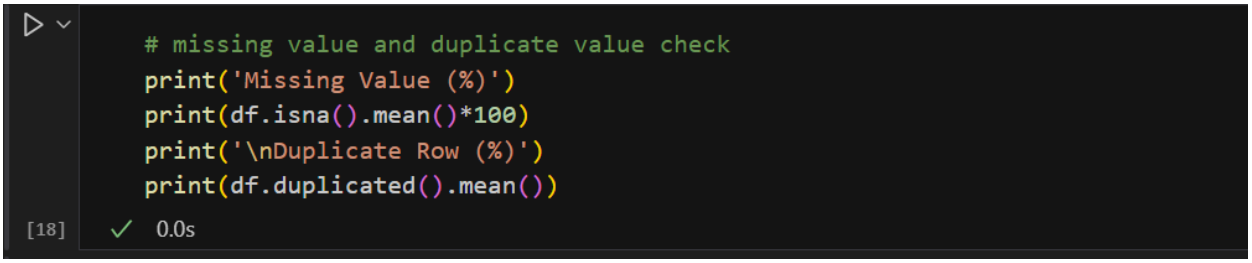
These libraries collectively support efficient data analysis, preprocessing, and preparation for predictive modeling.

2.0 Data Wrangling (Pandas/ Numpy)

2.1 Identifying and handling missing value

This code is used to examine the dataset for missing values and duplicate records. The function df.isna( ).mean( ) *100 calculates the percentage of missing values for each column, allowing verification of data completeness. The results show that all variables contain 0% missing values, indicating that no null data is present.

The function df.duplicated( ) is used to determine the proportion of duplicate rows in the dataset. Although the percentage of duplicate records is very small, the total number of duplicate rows is identified using df.duplicated( ). sum( )



```
Missing Value (%)
Country                          0.0
Age                              0.0
Gender                           0.0
Smoking_History                  0.0
Obesity                          0.0
Diabetes                         0.0
Chronic_Pancreatitis             0.0
Family_History                   0.0
Hereditary_Condition             0.0
Jaundice                         0.0
Abdominal_Discomfort             0.0
Back_Pain                        0.0
Weight_Loss                      0.0
Development_of_Type2_Diabetes    0.0
Stage_at_Diagnosis               0.0
Survival_Time_Months             0.0
Treatment_Type                   0.0
Survival_Status                  0.0
Alcohol_Consumption              0.0
Physical_Activity_Level          0.0
Diet_Processed_Food              0.0
Access_to_Healthcare             0.0
Urban_vs_Rural                   0.0
Economic_Status                  0.0
dtype: float64

Duplicate Row (%)
8e-05
```

To ensure data consistency, duplicate rows are removed using the drop_duplicates( ) function. The output confirms that four duplicate rows were detected before removal and that no duplicate rows remain after cleaning. This process ensures that the dataset is clean and ready for further preprocessing and analysis.

```
print("Number of duplicate rows BEFORE removal:", df.duplicated().sum())
df.drop_duplicates(inplace=True)
print("Number of duplicate rows AFTER removal:", df.duplicated().sum())
```

✓ 0.0s

```
Number of duplicate rows BEFORE removal: 4
Number of duplicate rows AFTER removal: 0
```

2.2 Data Formatting

This code standardises column names and categorical text values to ensure consistency and prevent errors during data processing. All column names are converted to lowercase using df.columns.str.lower( ) to maintain a uniform naming convention throughout the dataset.

The values in the gender column are converted to lowercase using string operations to eliminate inconsistencies caused by different text formats. Additionally, spaces in the stage_at_diagnosis column are replaced with underscores to create standardized category labels suitable for further encoding and analysis.

The printed output confirms that column names have been successfully standardised, the gender variable contains only two consistent categories (female and male), and the stage_at_diagnosis variable has been properly formatted into uniform categories (Stage_I, Stage_II, Stage_III, and Stage_IV). This data formatting step ensures that categorical variables are clean, consistent, and ready for encoding and machine learning processing.

```
# standardise column names and text values for stage_at_diagnosis
df.columns = df.columns.str.lower()
df['gender'] = df['gender'].str.lower()
df['stage_at_diagnosis'] = df['stage_at_diagnosis'].str.replace(' ', '_')

print(df.columns)
print(df['gender'].unique())
print(df['stage_at_diagnosis'].unique())
```

✓ 0.0s

```
Index(['country', 'age', 'gender', 'smoking_history', 'obesity', 'diabetes',
       'chronic_pancreatitis', 'family_history', 'hereditary_condition',
       'jaundice', 'abdominal_discomfort', 'back_pain', 'weight_loss',
       'development_of_type2_diabetes', 'stage_at_diagnosis',
       'survival_time_months', 'treatment_type', 'survival_status',
       'alcohol_consumption', 'physical_activity_level', 'diet_processed_food',
       'access_to_healthcare', 'urban_vs_rural', 'economic_status'],
      dtype='object')
['female' 'male']
['Stage_III' 'Stage_IV' 'Stage_II' 'Stage_I']
```

2.3 Data Normalization (centering/ scaling)

In this step, data normalization was applied to selected numerical features to ensure they are on a comparable scale before machine learning analysis. The StandardScaler from the Scikit-learn library was used to standardize the age and survival_time_months variables.

The fit_transform( ) function rescales the data by subtracting the mean and dividing by the standard deviation, resulting in features with a mean close to 0 and a standard deviation close to 1. This process helps prevent features with larger numeric ranges from dominating the learning process.

The descriptive statistics shown in the output confirm successful normalization, where both variables exhibit mean values approximately equal to zero and standard deviations close to one. This indicates that the numerical features have been properly centered and scaled, making them suitable for use in machine learning models.

```
# data normalization
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
df[['age', 'survival_time_months']] = scaler.fit_transform(
    df[['age', 'survival_time_months']]
)

# actual
print(df[['age', 'survival_time_months']].describe())

# rounded
desc = df[['age', 'survival_time_months']].describe()
desc.loc['count'] = desc.loc['count'].astype(int)
desc.round(10)
```

The results show that the normalization process was successfully applied to the age and survival_time_months variables. After standardization, both variables have mean

values approximately equal to zero and standard deviations close to one, which confirms effective centering and scaling.

The minimum and maximum values indicate that the normalized data now represents how far each observation deviates from the mean in terms of standard deviation units. For example, negative values represent observations below the mean, while positive values represent observations above the mean.

Overall, the normalized results demonstrate that numerical features have been placed on a consistent scale, reducing the influence of differing units and magnitudes. This improves the stability and performance of machine learning algorithms in subsequent modeling stages.

	age	survival_time_months
count	4.999600e+04	4.999600e+04
mean	3.612688e-16	-4.093054e-17
std	1.000010e+00	1.000010e+00
min	-3.463106e+00	-1.144306e+00
25%	-6.558025e-01	-7.007351e-01
50%	4.602341e-02	-3.458782e-01
75%	6.475885e-01	4.525496e-01
max	2.552545e+00	4.001118e+00

	age	survival_time_months
count	49996.000000	49996.000000
mean	0.000000	-0.000000
std	1.000010	1.000010
min	-3.463106	-1.144306
25%	-0.655803	-0.700735
50%	0.046023	-0.345878
75%	0.647588	0.452550
max	2.552545	4.001118

2.4 Binning

In this step, binning was applied to the age variable to group continuous values into meaningful categories. Since the age feature had been previously standardized using StandardScaler, the bin boundaries were defined based on scaled values.

The pd.cut( ) function was used to divide the normalized age values into three categories: Young, Middle-aged, and Elderly. Each data point was assigned to a corresponding age group based on its standardized value. This transformation converts a continuous numerical feature into a categorical variable, making it easier to analyze age-related patterns.

The output confirms that the binning process was applied successfully. Sample rows show the correct assignment of age values to their respective age groups. The frequency count indicates that the majority of patients fall into the Elderly category, followed by Middle-aged, while the Young group represents the smallest proportion of the dataset.

Overall, binning simplifies data representation and can help highlight age-related trends in pancreatic cancer prevalence during further analysis.

```
[47]  ✓ 0.0s

# binning age column
df['age_group'] = pd.cut(
    df['age'],
    bins=[-3, -1, 0, 3],   # scaled values after StandardScaler
    labels=['Young', 'Middle-aged', 'Elderly']
)

print(df[['age', 'age_group']].head(10))
print(df['age_group'].value_counts())
```

```
        age    age_group
0 -0.054237  Middle-aged
1  1.249154      Elderly
2  0.647588      Elderly
3 -0.856324  Middle-aged
4  1.750458      Elderly
5 -1.558150        Young
6  0.246545      Elderly
7 -0.856324  Middle-aged
8 -1.457889        Young
9 -1.157107        Young
age_group
Elderly        25158
Middle-aged    16994
Young           7776
Name: count, dtype: int64
```

2.5 Indicator Variables

In this step, categorical variables were converted into indicator variables to make them suitable for machine learning models. The pd.get_dummies( ) function was used to perform one-hot encoding on selected categorical features, including gender, treatment_type, stage_at_diagnosis, and urban_vs_rural

The parameter drop_first=True was applied to avoid multicollinearity by removing one reference category from each encoded variable. This ensures that the resulting dataset does not contain redundant features that could negatively affect model performance.

```
# indicator variable
df_encoded = pd.get_dummies(
    df,
    columns=['gender', 'treatment_type', 'stage_at_diagnosis', 'urban_vs_rural'],
    drop_first=True
)

print(df_encoded.columns)
print(df_encoded.head())
```

[48] ✓ 0.0s

The output shows that each category has been transformed into a separate binary column with values of True or False, indicating the presence or absence of a specific category. For example, variables such as gender_male, treatment_type_Surgery, and stage_at_diagnosis_Stage_IV represent individual category indicators.

Overall, this transformation enables categorical information to be represented numerically, ensuring compatibility with machine learning algorithms and preparing the dataset for model development in subsequent project stages.

```
Index(['country', 'age', 'smoking_history', 'obesity', 'diabetes',
       'chronic_pancreatitis', 'family_history', 'hereditary_condition',
       'jaundice', 'abdominal_discomfort', 'back_pain', 'weight_loss',
       'development_of_type2_diabetes', 'survival_time_months',
       'survival_status', 'alcohol_consumption', 'physical_activity_level',
       'diet_processed_food', 'access_to_healthcare', 'economic_status',
       'age_group', 'gender_male', 'treatment_type_Radiation',
       'treatment_type_Surgery', 'stage_at_diagnosis_Stage_II',
       'stage_at_diagnosis_Stage_III', 'stage_at_diagnosis_Stage_IV',
       'urban_vs_rural_Urban'],
      dtype='object')
         country       age  smoking_history  obesity  diabetes  \
0         Canada -0.054237                0        0         0
1   South Africa  1.249154                1        1         0
2          India  0.647588                0        0         0
3        Germany -0.856324                0        0         0
4  United States  1.750458                0        0         0

   chronic_pancreatitis  family_history  hereditary_condition  jaundice  \
0                     0               0                     0         0
1                     0               0                     0         0
2                     0               0                     0         0
3                     0               1                     0         1
4                     0               1                     0         0
```

```
   abdominal_discomfort  ...  access_to_healthcare  economic_status  \
0                     0  ...                  High              Low
1                     0  ...                Medium              Low
2                     0  ...                   Low           Middle
3                     0  ...                Medium           Middle
4                     0  ...                Medium              Low

     age_group  gender_male  treatment_type_Radiation  treatment_type_Surgery  \
0  Middle-aged        False                     False                    True
1      Elderly         True                     False                   False
2      Elderly        False                     False                   False
3  Middle-aged         True                      True                   False
4      Elderly        False                     False                   False

  stage_at_diagnosis_Stage_II stage_at_diagnosis_Stage_III  \
0                       False                         True
1                       False                         True
2                       False                        False
3                       False                        False
4                       False                        False

  stage_at_diagnosis_Stage_IV urban_vs_rural_Urban
0                       False                True
1                       False                True
2                        True               False
3                        True               False
4                        True               False

[5 rows x 28 columns]
```